ARIA

Agentic Research Intelligence Architecture

Project Overview · v4.1.0 · April 16, 2026 · Prepared by Ayush Chhoker

v4.1.0	18	30+
Current Version	Parallel Agents	Supported Sources

1. Executive Summary

ARIA (Agentic Research Intelligence Architecture) is a fully automated, AI-driven research collection and intelligence platform built in Python with a Flutter desktop UI. Given a topic and a research goal, ARIA dispatches up to 18 parallel web agents across academic databases, social platforms, news feeds, government portals, and more — then uses a large language model to clean, classify, and relevance-score every collected document before delivering structured exports in six formats.

The system is designed for researchers, analysts, and knowledge workers who need rapid, comprehensive literature reviews without manual searching. It runs equally well as a headless CLI tool or through its full-featured Flutter GUI, and supports both cloud LLMs (OpenAI, Anthropic, OpenRouter) and local models via Ollama.

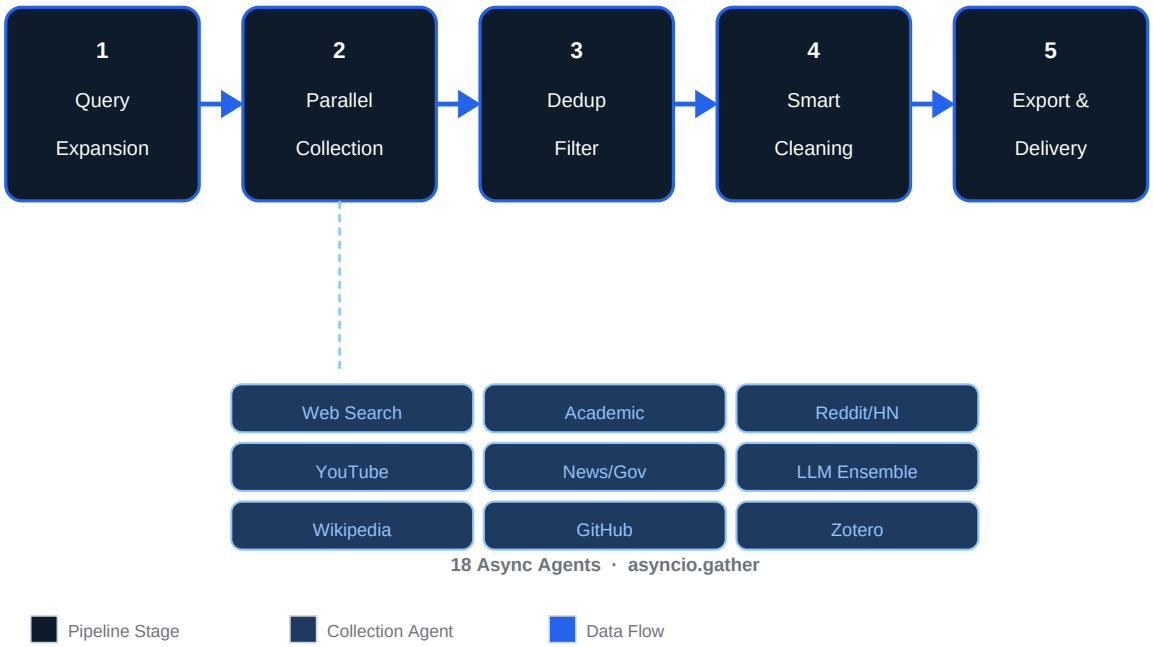
2. Project Status

Milestone	Status	Notes
Core async pipeline (v1)	■ Complete	18 collectors, asyncio.gather
FastAPI backend (v2)	■ Complete	REST API + Flutter UI
LLM ensemble + query expansion (v3)	■ Complete	Multi-model parallel queries

Milestone	Status	Notes
Smart Cleaning Agent (v4)	■ Complete	Auto content-type detection
Zotero integration	■ Complete	Local & cloud import modes
6-format export system	■ Complete	JSON, CSV, Markdown, ZIP
Packaging & PyPI distribution	■ In Progress	pyproject.toml ready
Enterprise SSO / auth layer	■ Planned	Future roadmap

■ 3. System Architecture

ARIA follows a five-stage pipeline architecture. Each stage is independently testable and configurable via mission YAML files.



Stage 1 — Query Expansion

An LLM expands the user's topic into 5–8 diverse search queries, ensuring broad coverage across different angles and terminology before any collection agent fires.

Stage 2 — Parallel Collection

18 async collector agents run simultaneously via asyncio.gather. Agents cover web search, academic databases (arXiv, PubMed, Semantic Scholar), Reddit, YouTube, Hacker News, Wikipedia, Stack Overflow, news RSS feeds, government portals, GitHub, Medium, Dev.to, Substack, multilingual search, LLM ensemble, and Zotero library import.

Stage 3 — Deduplication

Every collected document is fingerprinted using URL-hash and content-hash. Duplicates are discarded before storage, keeping the dataset clean and reducing downstream LLM processing costs.

Stage 4 — Smart Cleaning Agent

A single LLM call per document auto-detects content type across 10 categories (academic paper, news article, forum post, technical doc, etc.) and applies the matching structured extraction strategy. Each document receives a relevance score (0–1) against the user's research goal.

Stage 5 — Export & Delivery

Cleaned and raw results are materialized to disk and served through 6 download endpoints (JSON, CSV, Markdown, ZIP bundle). The Flutter UI presents a live-updating results table capped at 100 preview rows; full downloads bypass this cap.

4. LLM-Driven Data Extraction

ARIA uses LLMs at three distinct stages of the pipeline — not only for cleaning but as an active data extraction and generation layer. Each role is independently configurable and supports multiple providers (OpenAI, Anthropic, Ollama, OpenRouter).

Role 1 — Query Expansion (pre-collection)

Before any web agent fires, the *QueryExpander* sends the user's raw topic to an LLM and asks it to generate 5–8 semantically diverse search queries. This broadens coverage across different phrasings, sub-topics, and terminology, dramatically increasing recall compared to a single query.

Role 2 — LLM Ensemble as Live Source (collection)

The *LLMEnsembleAgent* queries multiple LLM models **in parallel** as first-class research sources. Each model's response is saved as a raw document (*source_type: 'llm_response'*) and flows through the normal deduplication and cleaning pipeline. When two or more models are enabled, an optional cross-model comparison synthesis pass produces *source_type: 'llm_comparison'* documents that highlight agreements and disagreements across models.

Role 3 — Structured Data Extraction per Document (cleaning)

The *LLMCleaner* (infocollector/cleaner/llm_cleaner.py) makes one LLM call per collected document. The prompt passes the raw text, URL, source type, and the user's research goal, and instructs the model to:

• Classify the document into one of 10 content types (academic_paper, news_article, forum_post, technical_doc, video_transcript, etc.)

• Score relevance against the research goal on a 0.0–1.0 float scale and set is_relevant: true when score ≥ 0.5

• Extract all fields declared in the mission's output_schema (e.g. methodology, key_findings, limitations for academic papers; headline stats and trends for data reports)

• Generate a 2–4 sentence summary of the document's main findings

Batch extraction runs with bounded concurrency (default 5 parallel LLM calls, configurable) and a 100 ms back-off between calls to stay within provider rate limits. If the LLM call fails or returns malformed JSON, the document falls back to a safe default (*cleaned: false*) and is still included in exports so no data is lost.

LLM Role	Module	Input	Output
Query Expansion	infocollector/cleaner/expander.py	User topic string	5–8 diverse search query strings
Ensemble Source	infocollector/ensemble/agent.py	Research topic + system prompt	Raw documents (source_type: llm_response / llm_comparison)
Structured Extraction	infocollector/cleaner/llm_cleaner.py	Raw document text + output_schema	content_type, relevance_score, schema fields, summary

5. Key Components

Component	Location	Responsibility
CLI Entry Point	aria/cli.py	Topic runs, API server, clean-only & resume modes
Orchestrator	aria/orchestrator/	Coordinates all pipeline stages end-to-end
Source Collectors	aria/sources/	18 async agents (web, academic, social, LLM, Zotero…)
Smart Cleaning Agent	infocollector/cleaner/	Content-type detection + structured extraction
LLM Ensemble Agent	infocollector/ensemble/	Multi-provider parallel LLM queries
FastAPI Backend	infocollector/api/	REST routes, run manager, request/response schemas
Access Manager	infocollector/access/	Credential handling, session mgmt, paywall fallbacks

Component	Location	Responsibility
Export Engine	aria/export/	JSON / CSV / Markdown / ZIP generation
Flutter UI	aria_studio/	Desktop/web GUI — config, live results, downloads
Mission System	missions/examples/	YAML-driven configuration for all pipeline parameters
Test Suite	tests/	pytest + asyncio — covers collectors, brain, API, output

6. Technology Stack

Layer	Technologies
Language & Runtime	Python 3.11+ · asyncio
Web Framework	FastAPI · Uvicorn
LLM Providers	OpenAI · Anthropic (Claude) · OpenRouter · Ollama (local)
Web Scraping	Playwright (Chromium) · BeautifulSoup4 · httpx · feedparser
Academic Sources	arXiv API · PubMed/Entrez · Semantic Scholar · yt-dlp
Reference Manager	pyzotero · Zotero local API
Data & Storage	JSON/CSV flat files · Optional Redis shared memory
Desktop UI	Flutter (Dart) — macOS / Windows / Web / Linux
Packaging	setuptools / pyproject.toml · pip
Testing	pytest · pytest-asyncio · black · isort · flake8 · mypy

7. Complete Data Source Directory

ARIA ships with 28 collector agents spanning 10 categories. Free sources are always active. Authenticated sources are silently skipped when credentials are absent — no configuration required.

Academic & Research

Collector	API / Method	What Is Extracted	Auth
arXiv	arXiv public API	Title, authors, abstract, PDF link, categories, date	None
PubMed	NCBI Entrez E-utilities	Title, authors, abstract, MeSH terms, PMID, journal	None
Semantic Scholar	S2 Graph API	Title, abstract, citation count, fields of study, DOI	None
IEEE Xplore	IEEE REST API v2	Full metadata, DOI, abstract, IEEE citations	API key
Elsevier ScienceDirect	Elsevier API	Abstract, author keywords, DOI, journal, volume	API key
Springer Nature	Springer API	Title, abstract, DOI, discipline, open-access flag	API key
JSTOR	Session cookie scrape	Title, abstract, journal, page range, JSTOR URL	Inst. login
ACM Digital Library	Session cookie scrape	Title, abstract, ACM DL link, conference/journal	Inst. login

Web & General Search

Collector	API / Method	What Is Extracted	Auth
Web Search	DuckDuckGo JSON / SerpAPI	Title, snippet, URL, source domain (multi-query expanded)	Optional
Web Scraper	Playwright + BeautifulSoup4	Full page text, title, meta description, outbound links	None
PDF Discovery	DuckDuckGo filetype:pdf + pdfminer	PDF text, title, page count, embedded metadata	None
Wikipedia	MediaWiki REST + Action API	Article text, summary, categories, infobox fields, related articles	None
GitHub	PyGithub API / DDG fallback	Repo description, README, stars, issues, topics, language	Optional
User URLs	Direct Playwright fetch	Full page content from explicitly provided URLs	None

News & Publishing

Collector	Sources / Method	What Is Extracted	Auth
News (RSS)	BBC, Reuters, AP, NPR, Guardian, Wired, Ars Technica — feedparser	Title, description, pub date, full article text	None
NewsAPI	newsapi.org REST API	Title, description, source, URL, publishedAt	API key
GNews	gnews.io REST API	Title, description, content, source name, pub date	API key
Medium	DDG site:medium.com + BS4 scrape	Article title, author, claps, full text, tags	None
Dev.to	dev.to public /articles API	Title, description, tags, reactions, reading time, body_markdown	None
Substack	RSS feedparser	Newsletter title, post title, body text, pub date	None
RSS / Atom	feedparser — any feed URL	Title, summary, content, author, pub date	None

Social & Community

Collector	API / Method	What Is Extracted	Auth
Reddit	reddit.com/search.json / PRAW	Post title, body, score, comments, subreddit, flair	Optional
Hacker News	Algolia HN search API	Title, URL, points, comment count, author, date	None
Stack Overflow	Stack Exchange API v2.3	Question title, body, accepted answer, tags, vote score	None
Twitter / X	Twitter API v2 (Bearer token)	Tweet text, author, likes, retweets, created_at, hashtags	Bearer token
LinkedIn	Playwright session auth + DDG fallback	Post text, author, company, reactions, reshares	Email + password
Slack	Slack Web API search.messages	Message text, channel, author, timestamp, thread replies	Bot token
Notion	Notion API v1 search endpoint	Page title, block content, last edited, workspace	Integration token

Media & Audio

Collector	API / Method	What Is Extracted	Auth
YouTube	YouTube Data API / yt-dlp	Video title, description, transcript (auto/manual), channel, views, tags	None
Podcast	RSS feedparser + OpenAI Whisper	Episode title, show description, full transcript (Whisper), pub date	Optional (Whisper)
Vimeo	Vimeo API v3	Video title, description, duration, tags, creator	Access token

Government, Reference & Knowledge Management

Collector	API / Method	What Is Extracted	Auth
Government (free)	data.gov catalog API + DDG site:.gov	Dataset title, description, agency, format, URL	None
Regulations.gov	regulations.gov API	Document title, docket ID, agency, comment period, PDF	Optional
Zotero (local)	Zotero desktop connector localhost:23119	Title, authors, abstract, PDF text, tags, collection	None
Zotero (cloud)	api.zotero.org via pyzotero	Same as local — entire library or specific collection	API key
Multilingual	DuckDuckGo 12 region codes (de/fr/es/zh/ja/pt/ar/ru/it/ko/nl/pl)	Same as web search — translated to English via LLM or deep_translator	None
LLM Ensemble	OpenAI / Anthropic / Ollama / OpenRouter / Mistral / Google / Together	Model-generated research content saved as source documents	API key

8. Intelligence & Analysis Features

Beyond collection and cleaning, ARIA includes a suite of advanced intelligence modules in *infocollector/intelligence/* that run automatically as part of the pipeline after documents are cleaned.

Smart Query Planning (infocollector/intelligence/query_planner.py)

An LLM decomposes the research topic into sub-questions and generates targeted search queries per source type (e.g., different queries for arXiv vs. Reddit vs. news feeds), maximising relevance per source.

Multi-dimensional Relevance Scoring (infocollector/intelligence/relevance_scorer.py)

Each document is scored across 5 axes: keyword overlap with the research goal, source authority (academic > news > social), recency, information density, and embedding cosine similarity. Final score is 0.0–1.0.

Semantic Deduplication (infocollector/intelligence/semantic_dedup.py)

Goes beyond URL/hash dedup. Uses MinHash LSH (datasketch) plus sentence-transformer embeddings to detect near-duplicate documents with different URLs — e.g., syndicated news articles or mirror sites.

Gap Detection (infocollector/intelligence/gap_detector.py)

After the first collection pass, an LLM analyses the collected documents and identifies unanswered sub-questions. A second-pass collection is automatically triggered with targeted queries to fill those gaps.

Knowledge Graph Extraction (infocollector/intelligence/knowledge_graph.py)

Extracts named entities (people, organisations, locations, policies) and their relationships from all documents. Detects contradictions across sources. Builds a NetworkX graph that can be exported for visualisation.

Image Extraction (infocollector/intelligence/image_extractor.py)

Extracts image URLs from HTML and page content. Filters noise (icons, logos, tracking pixels). Optionally runs LLM vision captioning on significant images to include visual context in the research output.

Streaming Pipeline (infocollector/intelligence/streaming.py)

The StreamingPipeline class processes documents as they arrive via an async bounded queue — cleaning happens in parallel with collection rather than in a separate batch phase, reducing total run time for large workloads.

Persistent Knowledge Base (infocollector/knowledge_base/kb.py)

SQLite database (~/.aria/aria_kb.db) that persists all collected documents across runs. Enables incremental collection: already-known URLs are skipped. Tracks relevance per topic and full run history. Supports stats queries.

Paywall Resolution (infocollector/access/)

When a paywalled academic URL is encountered, the access layer automatically queries Unpaywall (DOI-based) to find a free open-access version before skipping the document. Institutional session cookies can be provided for IEEE, Elsevier, Springer, JSTOR, and ACM.

Multi-language Translation (aria/brain/translator.py)

Documents collected in non-English languages are translated to English via LLM (for quality) with deep_translator (Google Translate) as a free fallback. The original-language text is preserved in *language_original* and *text_english* output fields.

9. Tool Overview & Use Cases

ARIA ships with four pre-built mission templates that cover the most common research workflows. Each template pre-configures sources, depth, and output schema — users only need to fill in the topic.

Mission Template	File	Best For	Key Sources	Sample Output Schema Fields
General Research	default.yaml	Any broad research topic	Web, Academic, YouTube, News, Wikipedia, HN	title, summary, key_topics, entities, document_type, relevance_score
Academic Research	academic_research.yaml	Literature reviews, systematic reviews	arXiv, PubMed, Semantic Scholar, IEEE, ACM, Springer, PDF	authors, abstract, methodology, key_findings, limitations, future_work, publication_venue
Competitive Intelligence	competitive_intel.yaml	Market research, competitor analysis	Web, News, YouTube, PDF	competitor_name, market_segment, strengths, weaknesses, opportunities, risks
Policy Research	county_ai_policies.yaml	Government documents, regulatory analysis	Government, PDF, Academic, Web, News (multi-lang)	jurisdiction, document_type, adoption_status, oversight_mechanisms, procurement_requirements

Who Is ARIA For?

Researchers & Academics: Run deep literature reviews across arXiv, PubMed, Semantic Scholar, IEEE, ACM, and Springer simultaneously. Export structured BibTeX-friendly JSON or CSV, or import an existing Zotero library as an additional source.

Policy Analysts: Collect government documents, regulations, and policy reports from data.gov, regulations.gov, and academic sources in multiple languages. The structured output schema captures jurisdiction, adoption status, and oversight mechanisms.

Market Intelligence Teams: Aggregate news, social signals (Reddit, Twitter, LinkedIn), and web content about competitors or markets. The competitive_intel mission template pre-configures strengths/weaknesses/opportunities/risks extraction.

Developers & Engineers: Headless CLI and FastAPI backend make ARIA scriptable. Use it in CI pipelines to automatically collect and structure documentation, Stack Overflow answers, GitHub issues, and technical blogs on a topic.

Data Scientists: The persistent Knowledge Base (SQLite) and 6-format export system make it easy to feed ARIA's output into downstream ML pipelines, vector databases, or fine-tuning datasets. Relevance scores and embeddings are included in exports.

10. Research Depth Levels

Level	Name	Max Documents	Deep Crawl
1	Surface	~50	None
2	Standard	~200	None
3	Deep	~500	1 BFS level
4	Thorough	~1,500	2 BFS levels
5	Maximum	5,000+	Full recursive

11. Configuration & Deployment

ARIA is configured through environment variables and YAML mission files. The mission file specifies the research topic, enabled sources, depth level, cleaning schema, and LLM provider. A reference mission template is provided in *missions/examples/default.yaml*.

Deployment Modes

Headless CLI: python -m aria.cli --topic "..." — ideal for scripting and CI pipelines.

API Server Mode: python -m aria.cli --serve-api — exposes a FastAPI server on port 8765 (configurable via ARIA_API_PORT). The Flutter UI connects to this server.

Flutter Desktop UI: aria_studio/ — cross-platform (macOS, Windows, Linux, Web). Provides a full GUI for configuration, live collection status, results browsing, and one-click downloads.

Resume & Recovery: --resume flag re-uses already-collected raw data, skipping re-scraping and jumping straight to the cleaning stage.

12. Smart Cleaning — Content Type Matrix

The Smart Cleaning Agent (introduced in v4.0) eliminates manual configuration. One LLM call per document identifies the content type and selects the extraction strategy automatically.

Content Type	Extracted Fields
academic_paper	Research question, methodology, findings, limitations
news_article	Who/what/when/where/why, key quotes, named sources
forum_post	Core question, consensus answer, vote/score signal

Content Type	Extracted Fields
social_media	Core claim, evidence offered, credibility signals
technical_doc	Concept, step-by-step instructions, code, version notes
video_transcript	Key points, named examples, speaker conclusions
encyclopedia	Definition, key facts, related concepts
opinion_essay	Thesis, evidence, counter-arguments, conclusion
gov_policy	Policy name, date, who is affected, penalties
data_report	Headline stats, trends, source, methodology

13. Export Formats

Group	Format	API Endpoint	Contents
Raw	JSON	GET /api/runs/{id}/export/raw	Full envelope — all docs + original text
Raw	CSV	GET /api/runs/{id}/export/raw/csv	Flat table — url, title, source, text (3000 chars)
Clean	JSON	GET /api/runs/{id}/export/clean	Structured array — all schema fields
Clean	CSV	GET /api/runs/{id}/export/clean/csv	One row per document, all schema columns
Clean	Markdown	GET /api/runs/{id}/export/clean/markdown	Human-readable report with titles & summaries
Bundle	ZIP	GET /api/runs/{id}/export/zip	clean/ + raw/ in all formats

14. Testing & Quality

The project includes a comprehensive pytest test suite covering all major subsystems. Tests are co-located in the *tests/* directory and organized to mirror the package structure.

- test_collectors — validates each async source agent independently
- test_agents — tests the LLM ensemble and query expander
- test_brain — covers orchestration logic and pipeline coordination
- test_output — verifies export formats and data integrity

- test_access — checks credential management and session handling

Code quality is enforced via black (formatting), isort (imports), flake8 (linting), and mypy (type checking) — all configured in pyproject.toml.

15. Roadmap

Near-term

- PyPI packaging and public release of aria-research
- Docker containerization for one-command deployment
- Incremental / scheduled research runs (delta mode)

Medium-term

- Vector database integration for semantic deduplication and retrieval
- Citation graph construction across collected papers
- Batch processing API for multi-topic research campaigns

Long-term

- Enterprise SSO and multi-user workspace support
- Plugin SDK for custom source collectors
- Fine-tuned cleaning model trained on ARIA's labeled output

Document prepared by Ayush Chhoker · chhokea@sunypoly.edu · April 16, 2026